

Este es el enunciado de la entrega que debo realizar, sin embargo, aunque el roadmap es bastante útil, creo que existe una forma más ordenada y que me ayude a entender mejor la implementación. Dame el orden en el que debo



Para entender el flujo completo sin bloquearte, la clave arquitectónica del enunciado es: el broker alimenta a `connector` (vía AMQP), `connector` le hace un `HTTP POST` a `master` (Koa), y `master` es el que guarda en PostgreSQL y expone los endpoints GET.

PDF + 2

Por eso sentías esa fricción: si construyes los endpoints de lectura (`/history`) sin datos reales en la BD, no tienes cómo probarlos adecuadamente.

A continuación tienes el orden de implementación estructurado de forma secuencial y testeable paso a paso.

Fase 1: Entorno Local y Base de Datos (Docker Compose)

Levanta primero la infraestructura de persistencia localmente para poder guardar y estructurar los datos antes de programar lógica de negocio compleja.

- **1. Definir `docker-compose.yml` base:**
 - Configura el servicio de base de datos (`postgres:15-alpine` o similar) con variables de entorno (`POSTGRES_DB` , `POSTGRES_USER` , `POSTGRES_PASSWORD`), puertos expuestos y volumen persistente.

PDF

- **2. Diseñar el esquema relacional en PostgreSQL:**
 - Analiza el JSON del enunciado: contiene campos a nivel raíz (`idpk` , `type` , `validUntil` , `metaContent` , `constraints`) y un arreglo de objetos `demands` (`city` , `demand` , `unit`).

PDF

- Debes incluir la columna `receivedAt` (TIMESTAMP en UTC).

PDF

- *Tip de diseño:* Puedes crear una tabla principal `events` y una tabla relacionada `demands` con llave foránea, o guardar el `packageBody` estructurado (o JSONB) asociando un identificador interno serial/UUID (`id`) autogenerado para cumplir con RF2.

PDF

Fase 2: El Servicio Web `master` (Koa.js)

Construye la API Koa enfocándote primero en la recepción y guardado de datos, y luego en la consulta.

- **3. Endpoint interno de ingesta (`POST /events` o similar):**
 - `connector` debe comunicarse con `master` vía HTTP POST.
- Implementa este endpoint para que valide el esquema JSON de la demanda, asigne la fecha actual en `receivedAt` , inserte el registro en PostgreSQL y devuelva un `201 Created` .
- *Prueba local:* Usa Postman, Thunder Client o `curl` enviando el JSON de ejemplo del enunciado. Verifica en Postgres que los datos persistan correctamente.
- **4. Endpoints de lectura (RF1, RF2, RF3, RF4):**
 - Con datos ya guardados en tu base local, implementa:
 - `GET /history` : Con paginación obligatoria por defecto (25 registros por página con `?page=1&limit=25`).
 - Filtrado dinámico en `GET /history` : Permitir filtros por `receivedAt` y demás atributos del evento.
 - `GET /history/:id` : Detalle de un evento según tu ID interno.
 - Endpoint `GET /health` básico para chequear disponibilidad posterior del container.

PDF

Fase 3: El `connector` (Node.js / amqplib)

Crea el worker que interactúa con el broker y hace de puente hacia `master` .

- **5. Script de conexión AMQP:**
 - Usa la librería `amqplib` conectándote a `broker.iic2173.org:5671` mediante AMQPS (puerto seguro) y tus credenciales asignadas.
 - Consume la cola asignada `observer.X.q` .

PDF

PDF

- **6. Forwarding HTTP & Resiliencia (RNF1):**

- Al consumir un mensaje, parsea el string JSON y envíalo mediante `fetch` o `axios` al endpoint interno de `master` (`POST http://master:3000/...`).
[PDF + 1](#)
- **Importante:** Envía el ACK (`channel.ack(msg)`) solo tras confirmar que `master` respondió con éxito.
- Implementa lógica de reconexión con reintentos exponenciales o `setTimeout` ante eventos `error` y `close` en la conexión de RabbitMQ, impidiendo que el proceso muera definitivamente si el broker cae.
[PDF](#)

Fase 4: Orquestación Total con Docker

Integra todos los componentes en tu máquina antes de tocar AWS.

- **7. Dockerfiles y Compose completo:**

- Crea `Dockerfile` para `master` y para `connector` .
- Define directivas `HEALTHCHECK` en ambos contenedores (para `master` vía `curl http://localhost:.../health` , para `connector` mediante un script o file-checking periódicamente actualizado al consumir o estar conectado).
[PDF](#)
- En `docker-compose.yml` , conecta `master` , `connector` y `postgres` en la misma red (`bridge`).
[PDF](#)
- Prueba el flujo E2E: levanta con `docker compose up` y verifica que `connector` lea del broker y `master` guarde en PostgreSQL sin intervención manual.

Fase 5: Infraestructura en AWS EC2 y DNS

Lleva el entorno testeado a producción.

- **8. Aprovisionamiento de EC2:**

- Lanza una instancia Ubuntu en la capa gratuita (t2.micro / t3.micro).
[PDF](#)
- Descarga y resguarda la llave `.pem` (no commitear al repo bajo ninguna circunstancia).
[PDF](#)
- En el Security Group, abre los puertos: `22` (SSH), `80` (HTTP), `443` (HTTPS).
[PDF](#)

- **9. Configuración de Dominio (RNF4):**

- Crea un registro de tipo `A` en tu proveedor de DNS apuntando a la Elastic IP o IP pública de tu EC2.
[PDF](#)

- **10. Despliegue de Docker:**

- Conéctate por SSH, instala Docker y Docker Compose.
PDF
- Clona tu repositorio y levanta tus 3 contenedores (`master` , `connector` , `postgres`). Expón únicamente el puerto de `master` hacia el host (ej. `127.0.0.1:3000`).
PDF

Fase 6: Reverse Proxy en el Host & Parte Variable

El enunciado exige explícitamente que Nginx esté instalado **directamente sobre la máquina EC2**, no dentro de Docker.

PDF

- **11. Instalación y configuración de Nginx (RNF3):**

- Instala Nginx en el sistema operativo del host (`sudo apt install nginx`).
PDF
- Configura un bloque de servidor con `proxy_pass [http://127.0.0.1:3000]` (`http://127.0.0.1:3000`); apuntando a tu subdominio/dominio.

- **12. Parte Variable (HTTPS o Balanceo):**

- *Si eliges HTTPS (Recomendada y acumulativa para entregas futuras):*
 - Instala Certbot (`sudo apt install certbot python3-certbot-nginx`).
PDF
 - Genera certificados Let's Encrypt y activa redirección automática HTTP -> HTTPS.
PDF
 - Asegura la tarea cron / systemd timer para el chequeo automático 2 veces al día (`certbot renew --dry-run`).
PDF
- *Si eliges Balanceo de Carga:*
 - Configura en `docker-compose` réplicas del servicio `master` en distintos puertos del host.
PDF
 - En el Nginx del host, define un bloque `upstream` balanceando el tráfico hacia esas instancias.
PDF

Fase 7: Entrega y Auditoría

- Llena el `README.md` con tu dominio, comando de conexión SSH con el `.pem` , y desglose de puntos logrados/no logrados.

PDF

- Verifica que los prompts/conversaciones de IA estén respaldados en la carpeta `ai-docs/`.
PDF
- Sube tu archivo `.pem` al buzón correspondiente en Canvas.
PDF

¿Tienes ya seleccionada la opción de la parte variable (HTTPS o Balanceo) para ayudarte a estructurar la configuración específica de Nginx y Docker?

qué es 15-alpine?

Es el **tag** de una imagen de Docker que combina dos cosas:

- **15:** La versión mayor del software principal (en este caso, **PostgreSQL 15**).
- **alpine:** El sistema operativo base en el que corre la imagen, que es **Alpine Linux**.

Alpine Linux es una distribución minimalista y ultraliviana (pesa alrededor de 5 MB de base). Al usar un tag con sufijo `-alpine`, la imagen resultante de Postgres termina pesando unos 80–90 MB en disco, en contraste con las imágenes estándar basadas en Debian/Ubuntu que suelen superar los 400 MB.

En servidores pequeños o con recursos ajustados como las instancias `t2.micro` / `t3.micro` de AWS Free Tier, las imágenes `-alpine` son la opción habitual para ahorrar memoria RAM, espacio en disco y acelerar las descargas (`docker pull`).

a qué se refiere con filtrado dinámico?

El **filtrado dinámico** significa que tu API no sabe de antemano qué filtros enviará el cliente en la URL (`ctx.query`), por lo que tu consulta a la base de datos debe armarse según los parámetros presentes en la petición en ese momento.

En lugar de crear una consulta estática fija, vas agregando condiciones `WHERE` a la consulta de PostgreSQL solo si el usuario incluye esos campos en los query parameters.

PDF

Casos de uso posibles

Un usuario podría consultar sin filtros:

```
GET /history
```

SQL resultante: `SELECT * FROM events LIMIT 25 OFFSET 0;`

O filtrar únicamente por fecha:

```
GET /history?receivedAt=2026-08-08
```

SQL resultante: `SELECT * FROM events WHERE received_at::date = '2026-08-08' LIMIT 25 OFFSET 0;`

PDF

O combinar múltiples filtros a la vez:

```
GET /history?receivedAt=2026-08-08&type=demand-set&city=Los Santos
```

SQL resultante: `SELECT * FROM events WHERE received_at::date = '2026-08-08' AND type = 'demand-set' AND ... LIMIT 25 OFFSET 0;`

PDF

Cómo se implementa en Node.js/Koa

Si usas SQL directo (vía `pg`), el patrón habitual es acumular las condiciones en un arreglo y usar consultas parametrizadas para evitar inyección SQL:

JavaScript



```
// ctx.query contiene los queryParams de la URL
const { receivedAt, type, idpk, page = 1, limit = 25 } = ctx.query;

const conditions = [];
const values = [];

if (receivedAt) {
  values.push(receivedAt);
  // Asume formato YYYY-MM-DD
  conditions.push(`received_at::date = ${values.length}`);
}

if (type) {
  values.push(type);
  conditions.push(`type = ${values.length}`);
}

if (idpk) {
  values.push(idpk);
  conditions.push(`idpk = ${values.length}`);
}
```

```
// Armar la cláusula WHERE solo si hay condiciones
const whereClause = conditions.length > 0 ? `WHERE ${conditions.join(' AND ')}` : '';

// Paginación obligatoria
const offset = (Number(page) - 1) * Number(limit);
values.push(Number(limit), offset);

const query = `
  SELECT * FROM events
  ${whereClause}
  ORDER BY received_at DESC
  LIMIT ${values.length - 1} OFFSET ${values.length}
`;

const result = await db.query(query, values);
```

Si decides utilizar un Query Builder u ORM (como Knex, Prisma o Sequelize), el concepto es el mismo: evalúas si el parámetro existe en `ctx.query` y encadenas los métodos `.where(...)` correspondientes antes de ejecutar la consulta.

En mi caso uso sequelize, a qué te refieres con encadenar los métodos `.where`?

En Sequelize no encadenas métodos `.where()` como en otros query builders (por ejemplo Knex), sino que **construyes dinámicamente el objeto que va dentro de la propiedad `where`** del método `findAll` o `findAndCountAll`.

Al decir "filtrado dinámico con Sequelize", nos referimos a inicializar un objeto `whereClause = {}` y asignarle llaves únicamente si el usuario mandó esos parámetros en `ctx.query`.

PDF

Ejemplo práctico en Koa con Sequelize

Imaginemos que tienes un modelo llamado `Event` y el usuario consulta `/history`:

PDF

JavaScript



```
const { Op } = require('sequelize');
const { Event } = require('../models');

async function getHistory(ctx) {
  // 1. Extraer queryParams de Koa
  const { page = 1, limit = 25, receivedAt, type, idpk } = ctx.query;

  const parsedPage = Math.max(1, parseInt(page, 10) || 1);
  const parsedLimit = Math.max(1, parseInt(limit, 10) || 25);
  const offset = (parsedPage - 1) * parsedLimit;

  // 2. Construir el objeto WHERE dinámicamente
  const where = {};

  // Si enviaron type en la URL, se agrega la condición
  if (type) {
    where.type = type;
  }

  // Si enviaron idpk en la URL, se agrega la condición
  if (idpk) {
    where.idpk = idpk;
  }

  // Si enviaron receivedAt (ej: ?receivedAt=2026-08-08)
  // Como en la BD suele ser TIMESTAMP/DATETIME, se filtra por rango de ese día completo
  if (receivedAt) {
    const startDate = new Date(`${receivedAt}T00:00:00.000Z`);
    const endDate = new Date(`${receivedAt}T23:59:59.999Z`);

    where.receivedAt = {
      [Op.between]: [startDate, endDate],
    };
  }

  // 3. Ejecutar la consulta pasando el objeto where resultante
  const { count, rows } = await Event.findAndCountAll({
    where,
    limit: parsedLimit,
    offset,
    order: [['receivedAt', 'DESC']],
  });

  // 4. Responder con los datos y metadata de paginación
  ctx.status = 200;
  ctx.body = {
```



```

totalItems: count,
totalPages: Math.ceil(count / parsedLimit),
currentPage: parsedPage,
limit: parsedLimit,
data: rows,
};
}

```

¿Qué ocurre por detrás?

- Si la URL es simplemente `/history?page=1&limit=25`, `where` es `{}` y Sequelize ejecuta:

SQL



```
SELECT * FROM "Events" ORDER BY "receivedAt" DESC LIMIT 25 OFFSET 0;
```

- Si la URL incluye parámetros (`/history?type=demand-set&receivedAt=2026-08-08`), `where` tendrá ambas claves y Sequelize armará automáticamente el SQL con `AND` :

SQL



```

SELECT * FROM "Events"
WHERE "type" = 'demand-set'
  AND "receivedAt" BETWEEN '2026-08-08 00:00:00' AND '2026-08-08 23:59:59.999'
ORDER BY "receivedAt" DESC LIMIT 25 OFFSET 0;

```

De esta manera, el endpoint soporta cualquier combinación de filtros sin duplicar código ni escribir consultas SQL manuales[cite: 1].

En el enunciado sale que: El endpoint `{url}/history` debe permitir filtrar las demandas por cada una de sus propiedades, especialmente los relacionados con tiempo

No, no es suficiente si solo filtras por `receivedAt`.

El enunciado es explícito en la redacción del RF4 (4p Esencial):

"El endpoint `{url}/history` debe permitir filtrar las demandas por cada una de sus propiedades, especialmente los relacionados con tiempo. Por ejemplo: `{url}/history?receivedAt=2026-08-08`"

PDF

La mención de `receivedAt` es únicamente un ejemplo, no el único requisito. Además, al ser un requisito marcado como **Esencial**, un fallo aquí pone en riesgo la revisión completa de la entrega.

PDF +1

¿Cuáles son las propiedades que debes poder filtrar?

Revisando el esquema JSON del enunciado, tienes propiedades a dos niveles:

PDF

1. Propiedades temporales (énfasis explícito del enunciado):

1. `receivedAt` (timestamp generado por tu sistema al recibirlo).
2. `validUntil` (campo temporal que viene dentro de `packageBody` en formato ISO8601 UTC).

PDF

2. Propiedades del evento (raíz):

1. `idpk` (UUID del evento).
2. `type` (ej. `"demand-set"`).

PDF

3. Propiedades de la demanda (`packageBody.demands`):

1. `city` (ej. `"New New york"`, `"Los Santos"`).
2. `unit` (ej. `"GW"`, `"W"`).
3. (Opcional/deseable) `demand` o rangos de demanda.

PDF

PDF

Cómo implementarlo en Sequelize según cómo diseñes tu base de datos

Dependiendo de cómo decidas guardar los datos en PostgreSQL, hay dos formas sencillas de soportar todos estos filtros:

Opción A: Relación 1 a N (Tablas `Events` y `Demands`)

Si tienes una tabla `Events` asociada con `Demands` (`Event.hasMany(Demand)`):

JavaScript



```
const { Op } = require('sequelize');

// Extraer parámetros soportados de la URL
const {
  receivedAt,
  validUntil,
  type,
  idpk,
  city,
  unit,
  page = 1,
  limit = 25,
} = ctx.query;

const eventWhere = {};
const demandWhere = {};

// 1. Filtros a nivel de Evento
if (idpk) eventWhere.idpk = idpk;
if (type) eventWhere.type = type;

// 2. Filtros temporales
if (receivedAt) {
  eventWhere.receivedAt = {
    [Op.between]: [
      new Date(`${receivedAt}T00:00:00.000Z`),
      new Date(`${receivedAt}T23:59:59.999Z`),
    ],
  };
}

if (validUntil) {
  // Puede ser por fecha exacta o rango de ese día
  eventWhere.validUntil = {
    [Op.between]: [
      new Date(`${validUntil}T00:00:00.000Z`),
      new Date(`${validUntil}T23:59:59.999Z`),
    ],
  };
}
```

```
// 3. Filtros a nivel de Demanda (si los mandan en la query)
if (city) demandWhere.city = city;
if (unit) demandWhere.unit = unit;

const results = await Event.findAndCountAll({
  where: eventWhere,
  include: [
    {
      model: Demand,
      as: 'demands',
      where: Object.keys(demandWhere).length > 0 ? demandWhere : undefined,
      required: Object.keys(demandWhere).length > 0, // INNER JOIN si filtra por ciudad
    },
  ],
  limit: parseInt(limit, 10),
  offset: (parseInt(page, 10) - 1) * parseInt(limit, 10),
  order: [['receivedAt', 'DESC']],
});
```

Opción B: Guardando el JSON en una columna `JSONB`

Si guardas el payload en una columna `data` de tipo `DataTypes.JSONB` en la tabla `Events`:

Sequelize permite filtrar directo sobre JSONB en Postgres:

JavaScript



```
if (city) {
  // En Postgres JSONB puedes consultar arrays de objetos
  eventWhere['packageBody'] = {
    demands: {
      [Op.contains]: [{ city }],
    },
  },
};
```

Resumen

Para asegurar el puntaje completo de **RF4** y evitar cualquier objeción en la corrección:

- Soporta al menos: `receivedAt`, `validUntil`, `idpk`, `type`, y `city`.
PDF + 4
- El filtrado por fecha (`receivedAt` y `validUntil`) debe admitir idealmente formato de día (`YYYY-MM-DD`) evaluando el rango del día completo o timestamps exactos.
PDF

ques eso de OP?

`Op` es un objeto que exporta Sequelize y significa **Operators** (Operadores).

Contiene símbolos que le indican a Sequelize qué operador de SQL debe utilizar en una consulta en lugar de una simple comparación de igualdad (`=`).

Por defecto, cuando escribes un objeto simple en Sequelize:

JavaScript



```
where: {  
  city: 'Los Santos'  
}
```

Sequelize asume igualdad estricta: `WHERE city = 'Los Santos'`.

Para qué sirve `Op`

Si necesitas hacer comparaciones más complejas (rangos, desigualdades, búsquedas de texto parcial o listas), usas las claves de `Op`:

- `[Op.eq]` : Igual (`=`)
- `[Op.ne]` : Distinto (`!=`)
- `[Op.gt]` : Mayor que (`>`)
- `[Op.gte]` : Mayor o igual que (`>=`)
- `[Op.lt]` : Menor que (`<`)
- `[Op.lte]` : Menor o igual que (`<=`)
- `[Op.between]` : Rango entre dos valores (`BETWEEN a AND b`)
- `[Op.in]` : Dentro de una lista (`IN (val1, val2)`)

- `[Op.iLike]`: Búsqueda no sensible a mayúsculas en PostgreSQL (`ILIKE` `'%texto%'`)

Ejemplo de importación y uso

Se importa directamente desde el paquete `sequelize`:

JavaScript



```
const { Op } = require('sequelize');

// Filtrar demandas mayores a 500
const results = await Demand.findAll({
  where: {
    demand: {
      [Op.gt]: 500 // SQL: WHERE demand > 500
    }
  }
});

// Filtrar eventos recibidos durante un día específico
const events = await Event.findAll({
  where: {
    receivedAt: {
      [Op.between]: ['2026-08-08 00:00:00', '2026-08-08 23:59:59']
      // SQL: WHERE "receivedAt" BETWEEN '2026-08-08 00:00:00' AND '2026-08-08 23:59:59'
    }
  }
});
```

Se usan los corchetes `[Op.between]` porque en JavaScript es la sintaxis para definir claves de objeto computadas a partir de un identificador de tipo `Symbol`.

```
const page = Math.max(1, parseInt(ctx.query.page, 10));
const limit = 25;
const offset = (page - 1) * limit;
```

si es de 25 no sería así?

Tiene dos detalles importantes a corregir:

- **1. Falta un valor por defecto para `page`** : Si el usuario no envía `?page=...`, `parseInt(undefined, 10)` evalúa a `NaN`. `Math.max(1, NaN)` devuelve `NaN`, lo que romperá el cálculo del `offset` y la consulta en PostgreSQL.
- **2. `limit` debe ser configurable por `queryParams`**: El RF3 dice: "La lista de demandas de electricidad debe estar paginada por default para que muestre cada 25 registros y poder cambiar de pagina cambiando un `queryParams`. Es decir: `{url}/history?page=2&limit=25`. Queda a criterio de ustedes si permiten traer más valores mediante otro número del `limit` en `queryParams`". Dejarlo fijo en `const limit = 25;` no lee el parámetro si el cliente lo manda explícitamente en la URL.

PDF +1

Forma correcta y robusta de escribirlo:

JavaScript



```
// Si no viene o es inválido, toma 1 para page y 25 por default para limit
const page = Math.max(1, parseInt(ctx.query.page, 10) || 1);
const limit = Math.max(1, parseInt(ctx.query.limit, 10) || 25);

const offset = (page - 1) * limit;
```

Comportamiento resultante:

- `/history` → `page = 1`, `limit = 25`, `offset = 0`

PDF

- `/history?page=2` → `page = 2`, `limit = 25`, `offset = 25`

PDF

- `/history?page=2&limit=10` → `page = 2`, `limit = 10`, `offset = 10`

PDF

- `/history?page=-5` → `page = 1`, `limit = 25`, `offset = 0` (protegido contra negativos)

ya pero me dicen que es cada 25 registros, por qué no simplemente que limit sea 25?

Si decides que el límite siempre sea 25, sí puedes dejarlo fijo como `const limit = 25;` y cumples perfectamente con el enunciado.

PDF

El enunciado dice:

"La lista de demandas de electricidad debe estar paginada por default para que muestre cada 25 registros y poder cambiar de pagina cambiando un queryParams. Es decir: `{url}/history?page=2&limit=25`. Queda a criterio de ustedes si permiten traer más valores mediante otro número del limit en queryParams."

PDF

Como dice *"queda a criterio de ustedes"*, tienes total libertad de ignorar otros números y forzar siempre 25 registros por página.

PDF

El único ajuste estrictamente necesario en tu código es proteger el caso en que `ctx.query.page` no venga en la URL (para que no te dé `NaN`):

JavaScript



```
const page = Math.max(1, parseInt(ctx.query.page, 10) || 1);
const limit = 25;
const offset = (page - 1) * limit;
```

Con esa sola línea del `|| 1`, tu lógica de 25 registros fijos queda lista y cumple con la pauta.

PDF

o sea que son max 10 paginas?

No, no hay un límite de 10 páginas.

El número de páginas dependerá de la cantidad de registros que existan en la base de datos dividida por 25:

$$\text{Total de páginas} = \left\lceil \frac{\text{Total de registros}}{25} \right\rceil$$

El enunciado advierte que el emisor de eventos enviará **miles de paquetes** durante el semestre y la corrección. Por ejemplo:

PDF

- Si tienes **2.500 registros**, tendrás **100 páginas** (`?page=1` hasta `?page=100`).

PDF

- Si tienes **10.000 registros**, tendrás **400 páginas**.

PDF

Lo que queda fijo en 25 es el **tamaño de cada página** (el número de elementos que devuelves en cada respuesta), mientras que el número de página (`page`) puede crecer tanto como registros tengas almacenados.

PDF

Explícame un poco el get health. A qué se refiere? es mi primera vez usando docker

El `GET /health` (o *Health Check*) es un endpoint ultra simple que sirve como un "chequeo de pulso médico" para saber si tu aplicación está viva, respondiendo y lista para recibir tráfico, o si se quedó pegada/congelada en memoria.

1. ¿Por qué es necesario con Docker?

Cuando ejecutas un contenedor Docker:

- Docker solo sabe si el proceso principal sigue corriendo en el sistema operativo.

- Pero una aplicación puede estar técnicamente "corriendo" y al mismo tiempo estar completamente colapsada (por ejemplo, bloqueada en un bucle infinito, sin conexión a la base de datos o sin responder peticiones HTTP).

Con la instrucción `HEALTHCHECK` que pide el enunciado (**RNF7 Esencial**), le ordenas a Docker:

PDF

"Cada X segundos, ejecuta una petición a este endpoint dentro del contenedor. Si responde con un código 200 OK, marca el contenedor como `(healthy)`. Si falla o da error repetidamente, márcalo como `(unhealthy)`".

PDF

2. ¿Cómo se implementa en tu servicio Koa (`master`)?

En tu API solo necesitas definir una ruta simple que devuelva un estado HTTP 200:

JavaScript



```
router.get('/health', async (ctx) => {
  ctx.status = 200;
  ctx.body = {
    status: 'ok',
    uptime: process.uptime(),
    timestamp: new Date().toISOString()
  };
});
```

(Opcional: Si quieres un healthcheck más robusto, puedes hacer un `await sequelize.authenticate()` rápido adentro para verificar que la base de datos también responda).

3. ¿Cómo lo configuras en Docker?

Tienes dos formas de declararlo (en el `Dockerfile` o en tu `docker-compose.yml`):

PDF

En el `Dockerfile` de `master` (usando `curl`)
:

Dockerfile



```
# Instalamos curl si la imagen es alpine
RUN apk add --no-cache curl

# Revisa la salud cada 30 segundos
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1
```

- `curl -f` : Falla (`exit 1`) si el código HTTP devuelto es 4xx o 5xx.
- `--start-period=10s` : Le da 10 segundos a la app para encender y conectarse a la BD antes de empezar a evaluarla.

O en tu `docker-compose.yml` :

YAML



```
services:
  master:
    build: ./master
    ports:
      - "3000:3000"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
```

4. ¿Y para el contenedor `connector` ?

El enunciado menciona explícitamente:

"Todos los containers deben tener una instrucción HEALTHCHECK que use curl o file checking para verificar que su servicio esté operativo y que pueda operar. No es necesario que connector exponga una API HTTP exclusivamente para este propósito."

Como `connector` es un script worker que consume de RabbitMQ y no necesita levantar un servidor web propio, puedes usar **file checking**:

1. En tu código JavaScript de `connector`, cada vez que recibe un mensaje o cada 10 segundos que sigue conectado al broker, actualiza la fecha de modificación de un archivo local (por ejemplo, con `fs.writeFileSync('/tmp/heartbeat', Date.now().toString())`).
2. En el `HEALTHCHECK` de Docker para `connector`, compruebas si ese archivo existe y si fue modificado recientemente:

Dockerfile



```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD test $(find /tmp/heartbeat -mmin -1) || exit 1
```

PDF

(Esto verifica que el archivo haya sido tocado hace menos de 1 minuto; si el worker se desconecta o se muere, deja de actualizar el archivo y Docker lo marca como `unhealthy`).

PDF

¿Cómo verás si funciona?

Cuando ejecutes `docker ps` en tu terminal, verás el estado en la columna `STATUS`:

- `Up 2 minutes (healthy)` → ¡Todo perfecto y listo para la corrección!
- `Up 2 minutes (unhealthy)` → Algo falló en la verificación.

Me surge la duda respecto a los services. Si tengo la api web en un mismo archivo, el connector va en ese mismo archivo o es una carpeta aparte, ej: en otro repositorio

No deben ir en el mismo archivo ni en repositorios separados. Deben ir en el **mismo repositorio**, pero organizados en **carpetas independientes**, ya que cada uno generará un contenedor Docker distinto.

PDF

El enunciado exige que `connector` y `master` corran como servicios independientes en contenedores separados dentro de la misma red Docker (RNF1 y RNF2).

PDF

Estructura de carpetas recomendada

En la raíz de tu repositorio, mantén una carpeta para cada servicio:

Plaintext



```
mi-proyecto/
├── docker-compose.yml
├── README.md
├── master/                                <-- Tu API en Koa
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       ├── index.js
│       ├── routes/
│       └── models/                        <-- Sequelize
├── connector/                             <-- Tu consumidor de RabbitMQ
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       └── index.js                      <-- Script que escucha AMQP y hace HTTP POST a master
```

Cómo se conectan en `docker-compose.yml`

Al declararlos como servicios separados, `connector` podrá comunicarse con `master` usando el nombre del servicio como si fuera un dominio interno:

PDF

YAML



```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: energyspark
```

```
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
  volumes:
    - pgdata:/var/lib/postgresql/data

master:
  build: ./master
  ports:
    - "3000:3000"
  depends_on:
    - postgres
  environment:
    DB_HOST: postgres
    PORT: 3000

connector:
  build: ./connector
  depends_on:
    - master
  environment:
    # connector le envía peticiones POST al contenedor master vía HTTP interno
    MASTER_URL: "http://master:3000/events"
    RABBITMQ_URL: "amqps://usuario:pass@broker.iic2173.org:5671"

volumes:
  pgdata:
```

Cada carpeta tendrá su propio `package.json` (así `connector` solo instala `amqplib` y `master` instala `koa`, `sequelize`, etc.) y su propio `Dockerfile` para construirse de forma aislada.

PDF

a qué refiere el enunciado con:

RNF2: (4p) Esencial

Su despliegue debe estar containerizado. Debe existir un servicio dentro de un container llamado



El requisito **RNF2** se refiere a tres condiciones técnicas concretas sobre cómo deben comunicarse y aislarse tus aplicaciones en Docker:

- **1. Cada componente corre en su propio contenedor:** Ni la API ni el worker deben ejecutarse directo en la máquina EC2 (salvo Nginx). La API en Koa debe estar empaquetada dentro de un contenedor dedicado llamado explícitamente `master`.
PDF + 1
- **2. Comunicación interna a través de una red Docker (Docker Network):** Ambos contenedores (`master` y `connector`) deben pertenecer a la misma red virtual de Docker. Gracias al DNS interno de Docker, `connector` no necesita usar IPs cambiantes ni salir a internet para hablar con `master`: basta con apuntar a `http://master:PUERTO`.
PDF + 1
- **3. `master` debe exponer un endpoint HTTP POST receptor:** La API debe tener una ruta (por ejemplo, `POST /events` o `POST /demands`) lista para recibir los JSONs que `connector` le envíe desde su contenedor.
PDF

Cómo se refleja esto en tu `docker-compose.yml`

Cuando levantas servicios con Docker Compose, por defecto se crea una red compartida (`default`) para todos ellos. Sin embargo, para cumplirlo de forma explícita y ordenada, se define una red compartida con los nombres exactos:

YAML



```
version: '3.8'

networks:
  app-network:
    driver: bridge

services:
  master:
    container_name: master          # Cumple con el nombre solicitado
    build: ./master
    networks:
      - app-network                # Ambos en la misma network docker
    ports:
      - "3000:3000"               # Para que el Nginx del host pueda acceder a él
    environment:
      - PORT=3000

  connector:
    container_name: connector
    build: ./connector
    networks:
      - app-network                # Misma network que master
    environment:
```

```
# connector se comunica con master usando su nombre de host interno en la red
- MASTER_URL=http://master:3000/events
depends_on:
- master
```

El flujo exigido por RNF2

1. RabbitMQ entrega el mensaje a `connector`.
[PDF](#)
2. `connector` procesa el mensaje y hace un `POST http://master:3000/events` viajando por `app-network`.
[PDF](#)
3. El contenedor `master` atiende la petición, extrae el body y lo guarda en la base de datos.
[PDF](#)

dame una petición de prueba para probar el master

Aquí tienes una petición de prueba utilizando exactamente la estructura y campos del esquema JSON que indica el enunciado:

[PDF](#)

Comando con `curl` (para probar desde tu terminal)

Bash



```
curl -X POST http://localhost:3000/events \
-H "Content-Type: application/json" \
-d '{
  "idpk": "d3b07384-d113-494a-a0e8-032049141f2a",
  "type": "demand-set",
  "packageBody": {
    "demands": [
      {
        "city": "New New york",
        "demand": 1013.123,
        "unit": "W"
      },
      {

```



```

        "city": "Los Santos",
        "demand": 10223,
        "unit": "GW"
      }
    ],
    "validUntil": "2026-12-12T00:00:00Z",
    "metaContent": "batch-energy-sample-01",
    "constraints": {}
  }
}'

```

(Nota: Si nombraste a tu ruta de otra forma, cambia `/events` por la ruta correspondiente).

Body en formato JSON crudo (para Postman / Thunder Client)

Si usas una herramienta gráfica, configura el método como `POST`, la URL a `http://localhost:3000/events`, en los headers añade `Content-Type: application/json`, y en el **Body** (raw JSON) pega lo siguiente:

PDF

JSON



```

{
  "idpk": "d3b07384-d113-494a-a0e8-032049141f2a",
  "type": "demand-set",
  "packageBody": {
    "demands": [
      {
        "city": "New New york",
        "demand": 1013.123,
        "unit": "W"
      },
      {
        "city": "Los Santos",
        "demand": 10223,
        "unit": "GW"
      }
    ]
  },
  "validUntil": "2026-12-12T00:00:00Z",
  "metaContent": "batch-energy-sample-01",
  "constraints": {}
}

```

Lo que debe pasar tras esta petición en `master`:

1. **Respuesta HTTP:** Debería devolver un código `201 Created` o `200 OK`.
2. **Registro de fecha:** `master` debe generar e insertar el campo `receivedAt` con el timestamp UTC del momento en que procesó la petición (`new Date()` o `CURRENT_TIMESTAMP`).
3. **Persistencia:** Al consultar después tu endpoint de lectura:

Bash



```
curl http://localhost:3000/history
```

deberías ver este registro devuelto en la lista con su ID interno y todos sus campos asociados.

PDF